

How to sample from a discrete distribution? [duplicate]

Asked 10 years, 1 month ago Modified 3 years, 3 months ago Viewed 61k times



38



This question already has answers here:

[How to generate numbers based on an arbitrary discrete distribution?](#) (7 answers)

Closed 7 years ago.

Assume I have a distribution governing the possible outcome from a single random variable X . This is something like $[0.1, 0.4, 0.2, 0.3]$ for X being a value of either 1, 2, 3, 4.

Is it possible to sample from this distribution, i.e. generate pseudo random numbers upon each of the possible outcomes given the probability of that outcome. So if I wanted to know what the probability of getting a 2 is, the sampling operation may return 0.34 or something like that.

The reason I ask is that I'm trying to implement an action selection policy for a reinforcement learning method based on a research paper. From what I gather from the paper, the author is able to sample the distribution by "mapping the uniform distribution $U[0,1]$ through cumulative probability density functions obtained by adaptive numerical integration". From this he then samples the transition probabilities for each trial...

I would be grateful for any info on this...

Thanks in advance

probability

distributions

random-generation

weighted-sampling

Share Cite Edit Follow Flag

edited Jan 20, 2015 at 6:44



Glen_b

275k

37

605

1005

asked Aug 20, 2013 at 20:40



Barry

523

1

5

6

1 There are a variety of methods of sampling discrete probability distributions. The paper uses the cdf (generate a uniform, $U = u$ on $(0,1)$, if $u < 0.1$ output "1", if it's $< 0.1 + 0.4$ output "2" and so on). There are far more efficient methods if speed is an issue (e.g. if you wanted to sample billions of times). – Glen_b Aug 20, 2013 at 21:37



@Glen_b could you please name more efficient methods for sampling a discrete rv? This is very interesting. – Riga Aug 21, 2013 at 11:13



@Riga see my answer below – Glen_b Aug 22, 2013 at 3:18

- 2  There's a nice article explaining the "alias method" here: keithschwarz.com/darts-dice-coins
 – Danica Aug 22, 2013 at 4:17

5 Answers

Sorted by: Highest score (default)



Sure. Here's an R function that will sample from that distribution n times, with replacement:

39

```
sampleDist = function(n) {
  sample(x = c(1,2,3,4), n, replace = T, prob = c(0.1, 0.4, 0.2, 0.3))
}

# > sampleDist(10)
# [1] 4 2 2 2 2 2 4 1 2 2
```



If you want to go a little lower level, you can see the actual algorithm used by checking out the R source (written in C):



```
/* Unequal probability sampling; with-replacement case
 * n are the lengths of p and perm. p contains probabilities, perm
 * contains the actual outcomes, and ans contains an array of values
 * that were sampled.
 */

static void ProbSampleReplace(int n, double *p, int *perm, int nans, int *ans)
{
    double rU;
    int i, j;
    int nm1 = n - 1;

    /* record element identities */
    for (i = 0; i < n; i++)
        perm[i] = i + 1;

    /* sort the probabilities into descending order */
    revsort(p, perm, n);

    /* compute cumulative probabilities */
    for (i = 1; i < n; i++)
        p[i] += p[i - 1];

    /* compute the sample */
    for (i = 0; i < nans; i++) {
        rU = unif_rand();
        for (j = 0; j < nm1; j++) {
            if (rU <= p[j])
                break;
        }
        ans[i] = perm[j];
    }
}
```

Share Cite Edit Follow Flag

edited Oct 22, 2015 at 17:42



hlin117

209 1 11

answered Aug 20, 2013 at 21:19



jtobin

1,526 8 9

▲ Ok I understand now what is happening, thanks so much for all the answers, I really hope that this will help someone else. I wish I could have selected all right answers. Thanks to everyone – **Barry**
 Aug 20, 2013 at 22:14

2 ▲ Why do you have to sort the discrete distribution? – **Pavithran Iyer** Dec 8, 2016 at 16:31

▲ @PavithranIyer I don't (or didn't) see it either. Maybe it's an optimization attempt: Testing for the larger probabilities first means you can more often stop early in the loop. But I doubt that's worth the cost of first sorting, unless you sample very often (large n ans). But then again, if you only sample a few times, you won't notice the difference. – **lucidbrot** Apr 3, 2018 at 14:05 ✎



In response to a question in comments, here's an outline of a few potentially* faster ways to do discrete distributions than the cdf method.

18



* I say "potentially" because for some discrete cases a well implemented inverse-cdf approach can be very fast. The general case is harder to make fast without introducing additional tricks.



For the case of four different outcomes as in the example in the question, the naive version of the inverse cdf approach (or effectively equivalent approaches) are fine; but if there are hundreds (or thousands, or millions) of categories it can become slow without being a bit smarter (you certainly don't want to be *sequentially* searching through the cdf until you find the first category in which the cdf exceeds a random uniform). There are some faster approaches than that.

[You could see the first few things I mention below s having a connection to faster-than-sequential approaches to locating a value using an index and so are in a way just a "smarter version of using the cdf". One can of course look at "standard" approaches to solve related problems like "searching a sorted file", and end up with methods with much faster than sequential performance; if you can call suitable functions, such standard approaches may often be all you need.]

Anyway, to some efficient approaches for generating from discrete distributions.

1) the "Table method". Instead of being $O(n)$ for n categories, once set up, the "simple" version of this in (a) (if the distribution is suitable) is $O(1)$.

a) *Simple approach* - assuming rational probabilities (done on the above data example):
 - set up an array with 10 cells, containing a '1', four '2's, two '3's and three '4's. Sample that using a discrete uniform (easy to do from a continuous uniform), and you get simple fast code.

b) *More complex case* - doesn't need 'nice' probabilities. Use 2^k cells, or rather, you'll end up using a few less than that. So for example, consider the following:

x	0	1	2	3	4	5	6
P(X=x)	0.4581	0.0032	0.1985	0.3298	0.0022	0.0080	0.0002

(We could have 10000 cells and use the previous exact approach, of course, but what if these probabilities are irrational, say?)

Let's use $k = 8$. Multiply the probabilities by 2^k and truncate to find out how many of each cell type we need:

x	0	1	2	3	4	5	6	Tot
$P(X=x)$	0.4581	0.0032	0.1985	0.3298	0.0022	0.0080	0.0002	1.0000
$[256p(x)]$	117	0	50	84	0	2	0	253

Then the last 3 cells are basically "generate instead from this other distribution" (i.e. $p(x) - \frac{\lfloor 256 p(x) \rfloor}{256}$ normalized to a p.m.f.):

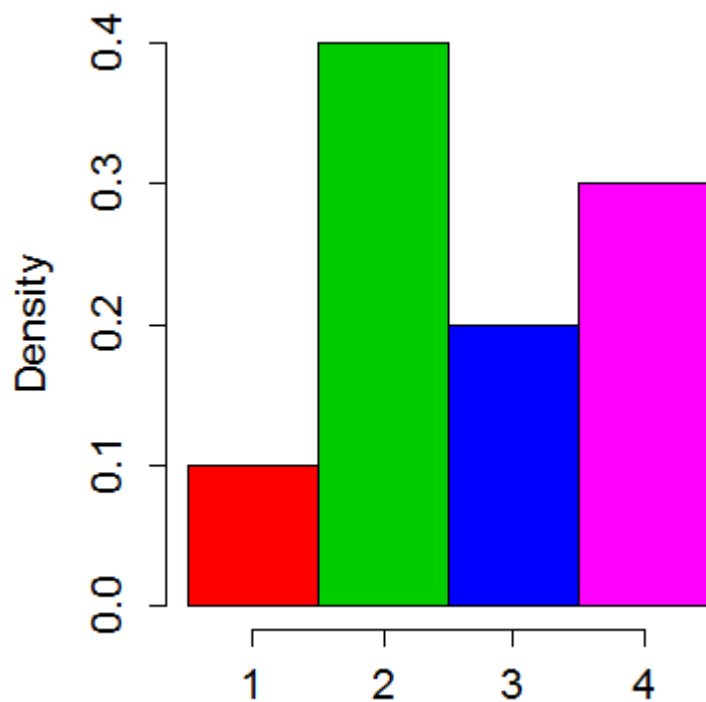
x^*	0	1	2	3	4	5	6	Tot
$P(X=x^*)$	0.091200	0.273067	0.272000	0.142933	0.187733	0.016000	0.017067	1.000000

The "spillover" table can be done by any reasonable method (you only get here about 1% of the time, it doesn't need to be as fast). So $\frac{253}{256}$ of the time we generate a random uniform, use its first 8 bits to pick a random cell, and output the value in the cell; after the initial setup all of this can be made very fast. The other $\frac{3}{256}$ of the time we hit a cell that says "generate from the second table". Almost always, you generate a single uniform on $(0, 1)$ and get a discrete random number from a multiply, a truncation and the cost of accessing an array element.

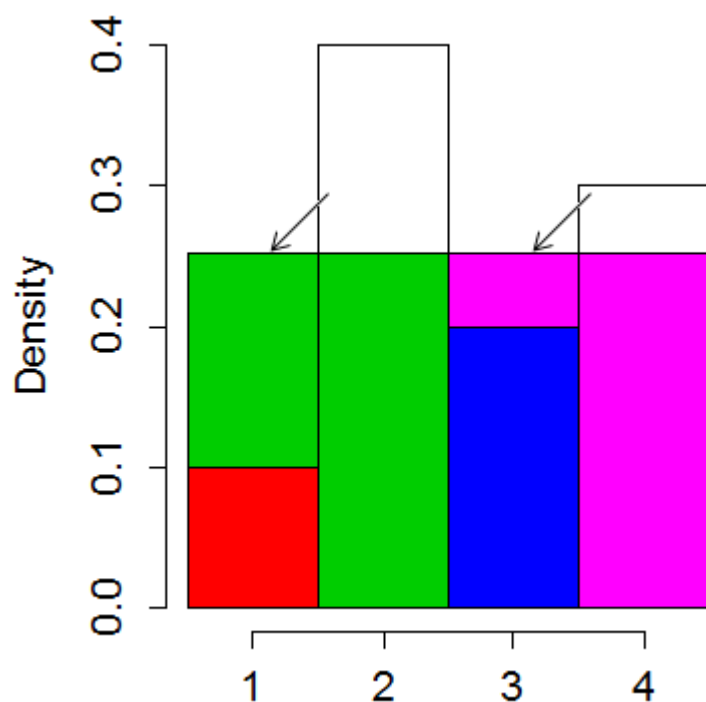
2) "Squaring the histogram" method; this is kind of related to (1), but each cell can actually generate either one of *two* values, depending on a (continuous) uniform. So you generate a discrete value from 1 to n , then within each one, check whether to generate its main value or its second value. It works with bounded random variables. There's no spillover table, and it generally uses much smaller tables than method (1). Usually, it's set up so that the choice of $1:n$ uses the first few bits of a uniform random number, and the remainder of it tells you which of the two values for that bin to output.

Perhaps the easiest way to outline the method is to do it on the above example:

Think of the distribution as a histogram with 4 bins:



We cut off the tops of the tallest bars and put them in the shorter ones, 'squaring it off'. The average 'height' of a bar will be 0.25. So we cut 0.15 off the second bar and put it in the first and 0.05 off the fourth and put it in the third:



Its always possible to organize this in such a way that no bin ends up with more than 2 colors, though one color may end up in several bins.

So now you choose one of the 4 bins at random (requires 2 random bits off the top of a uniform). You then use the remaining bits to specify a uniformly distributed vertical position and compare with the break between colors to work out which of two values to output. While very fast it's usually not quite as fast as the 'table' method.

--

These methods can be adapted to deal with unbounded variables, where again, it's 'mostly fast'.

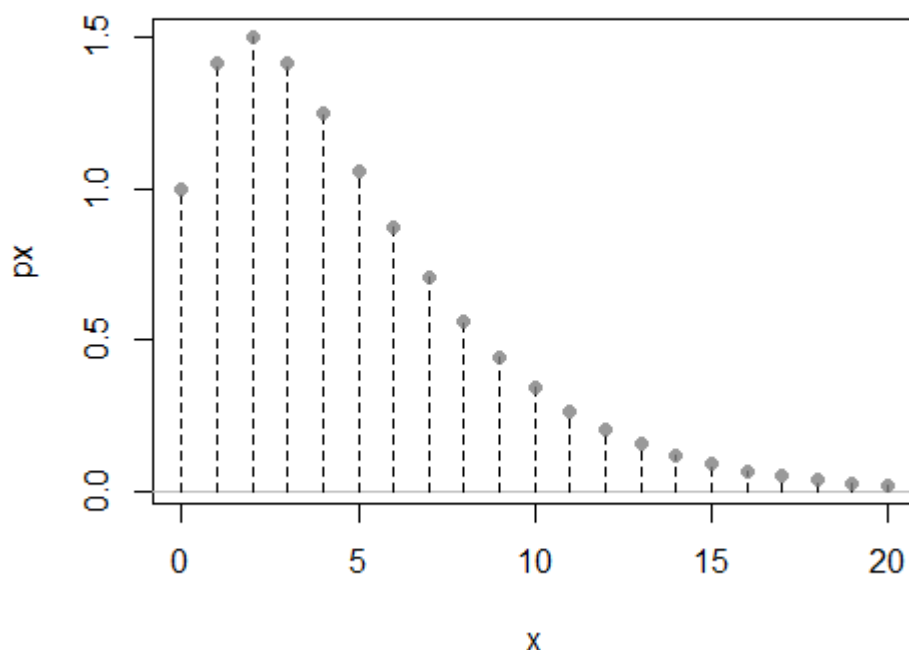
A reference: <http://www.jstatsoft.org/v11/i03/paper>

The relatively slow part of these is creating the tables of values; they're suitable when you know what you're going to be generating ("we need to sample values from this distribution many times in the future") rather than trying to create it as you go. "We need to sample a million values from this ASAP, but we'll never need to do it again" creates different priorities; in many situations some of the "standard computing approaches" to looking up sorted values (i.e. to doing the cdf method more quickly) may actually be about your best choice.

There are still other fast approaches to generating from discrete distributions. Carefully coded, you can do some very fast generation. For example:

3) the rejection method ("accept-reject") can be done with discrete distributions; if you have a discrete majorizing function ("envelope") which is a scaled-up discrete pmf that you can already generate from in a fast way, it adapts directly, and in some cases can be very fast. More generally you can take advantage of being able to generate from continuous distributions (for example by discretizing the result back to a discrete envelope).

Here imagine that we have some discrete probability function f for which we don't have a convenient cdf (or inverse-cdf) -- indeed in this illustration we didn't even have the normalizing constant, so our plot is unnormalized:



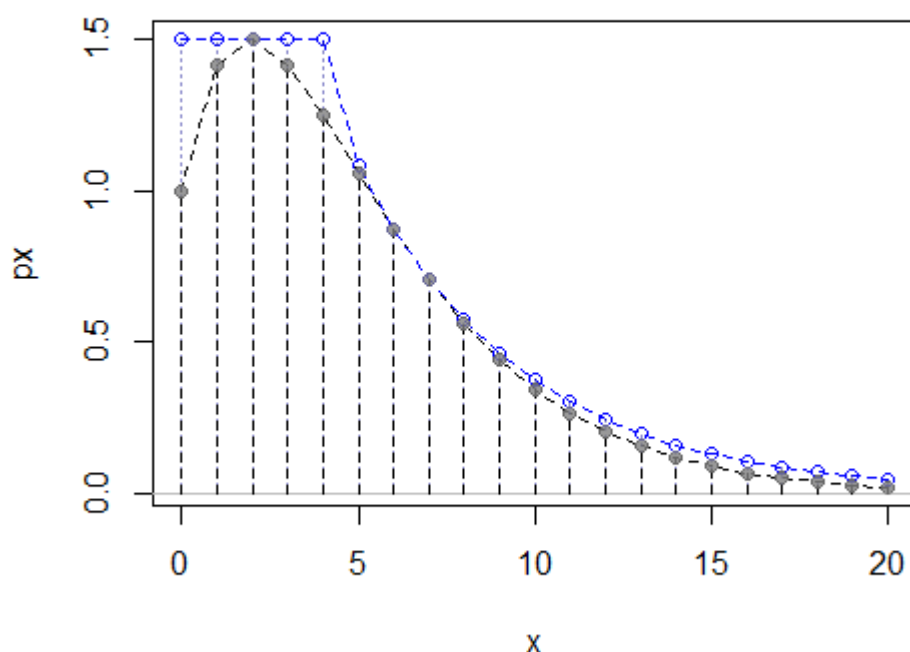
Now we need to find some convenient-to-generate-from discrete probability function g , which can be multiplied by a constant c and be everywhere at least as large as f (we need to

be sure that this remains true for all x values). That is, $c \cdot g(x) \geq f(x)$ for all the possible x values.

Sometimes a suitable g is easily identified, but one useful option is to take a mixture of a discrete uniform for the left part, and a distribution with at least as heavy a tail as f on the right. Two reasonably convenient choices for that are a geometric distribution (when the tail isn't decreasing slower than exponentially) and something like a discretized Pareto or discretized half-Cauchy distribution, obtained by taking $\lfloor X \rfloor$ for some Pareto or half-Cauchy random variate X (in either case for when the pmf is decreasing slower than exponentially).

(For that matter, the geometric itself can be generated by discretizing an exponential.)

In this case, a discrete uniform on the left and a geometric on the right work quite well:



(Reminder: what is plotted here is an unnormalized pmf, so the y-axis doesn't represent probability but something proportional to probability)

Then the procedure is a matter of simulating a proposed value x from g , simulating a uniform, U on $(0, c \cdot g(x))$ and if $U < f$, accepting that proposed x (otherwise rejecting it and generating a new proposed x).

Share Cite Edit Follow Flag

edited Jun 9, 2020 at 0:57

answered Aug 22, 2013 at 3:17



Glen_b

275k

37

605

1005



Thank you, Glen! 2^k approach is promising. Could you please clarify the "Squaring the histogram" method by providing an example? – Riga Aug 22, 2013 at 7:35

- 1  Sure, I can have a go, though recall that your original comment asked me to *name* approaches... and that's what its name is. In the meantime, there's a good explanation of the basic process [here](#). It's also called the [Robin Hood method](#). – Glen_b Aug 22, 2013 at 7:41 

 @Riga I updated the explanation with a brief outline of the idea for the second case on the example in the question. – Glen_b Aug 22, 2013 at 23:51

 Thank you Glen for your time! Your references and explanation is a valuable piece of knowledge. – Riga Aug 23, 2013 at 11:32

- 1  @greendiod It's $p(x) - \frac{\lfloor 256p(x) \rfloor}{256}$ scaled up to sum to 1, so in the example, that would be multiplied by 256/3. – Glen_b Jul 6, 2016 at 18:57

 In python you could do something like

7

```
from scipy.stats import rv_discrete

x=[1,2,3,4]
px=[0.1,0.4,0.2,0.3]

sample=rv_discrete(values=(x,px)).rvs(size=10)
```

 Which would give you 10 samples from the distribution. You could repeat this then find the proportions of samples that are 2.

Share Cite Edit Follow Flag

answered Aug 20, 2013 at 20:51

 **user1893354** 1,855 4 18 27

 Yes it is possible and fairly easy, exactly how depends on what tool(s) you are using.

6 In R it would be `sample(1:4, n, prob=c(0.1,0.4,0.2,0.3), replace=TRUE)` where `n` is the number of values you want to sample.

 In tools without an equivalent function you can still generate a uniform value and then your RV will equal 1 if it is below 0.1, 2 if it is between 0.1 and 0.5, 3 if between 0.5 and 0.7, and 4 if greater than 0.7 (that is the idea of mapping to the cumulative).

 For your example you could also sample uniformly from a set with one 1, four 2's, two 3's, and three 4's to get the same probabilities.

Share Cite Edit Follow Flag

answered Aug 20, 2013 at 21:22

 **Greg Snow** 50.2k 2 108 183

 +1. As a working example of the equivalent function in Excel (just to show how simple this is), set up an array of cumulative sums of the probabilities (e.g., (0,0.1,0.5,0.7,1.0)), give it a name (e.g., `cumsum`), and use `=MATCH(RAND(),cumsum,1)` to generate values 1,2,3,4 with probabilities 0.1, 0.4,

0.2, and 0.3 respectively. This clearly shows how weighted sampling is related to array searches.
– [whuber](#) ♦ Aug 20, 2013 at 22:01

▲ Thanks so much for this answer it really helped me a lot, I really hope that this will help someone else. I wish I could have selected all right answers. Thanks to everyone – [Barry](#) Aug 20, 2013 at 22:15



In Stata:

2

In Mata use `rdiscrete()` as documented at http://www.stata.com/help.cgi?mf_runiform



In Stata itself, there are various ways. Here's one:



```
. gen rnd = runiform()  
. gen y = cond(rnd <= 0.1, 1, cond(rnd <= .5, 2, cond(rnd <= .7, 3, 4)))
```



Share Cite Edit Follow Flag

answered Aug 20, 2013 at 21:40



[Nick Cox](#)

53.2k

8

121

175